

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Emna Kaaniche

Année Universitaire : 2025-2026

Table des matières

Introduction Générale	2
Présentation du Dataset	2
1 Prétraitement des Données (Preprocessing)	3
1.1 Introduction	3
1.2 Nettoyage initial et Feature Engineering	3
1.3 Prévention de la Fuite de Données (Train/Test Split)	3
1.4 Traitement des valeurs manquantes et aberrantes	3
1.5 Encodage des variables catégorielles	3
1.6 Mise à l'échelle (Standardization)	4
1.7 Conclusion	4
2 Modélisation	5
2.1 Introduction	5
2.2 Choix de l'algorithme : Random Forest	5
2.3 Justification du paramétrage : Pourquoi 200 arbres ?	5
2.4 Le défi du Data Leakage (Fuite de données)	5
2.5 Évaluation et Résultats définitifs	6
2.5.1 Validation Croisée et Stabilité	6
2.6 Analyse de l'importance des variables	6
2.7 Conclusion du chapitre	6

Introduction Générale

Le secteur du e-commerce est marqué par une concurrence accrue où la fidélisation des clients est devenue un enjeu stratégique majeur. Dans ce contexte, ce projet porte sur l'analyse et la prédiction du comportement des clients d'une boutique de vente de cadeaux en ligne.

L'objectif principal est de développer un modèle de Machine Learning capable d'anticiper le **Churn** (l'attrition), c'est-à-dire d'identifier les clients qui risquent de ne plus effectuer d'achats sur la plateforme.

Objectifs du Projet

- **Exploration** : Comprendre les habitudes d'achat et les profils types des clients.
- **Prédiction** : Anticiper le départ des clients grâce à des algorithmes de classification.
- **Actionnabilité** : Fournir des indicateurs permettant à l'entreprise de mettre en place des campagnes de rétention ciblées.

Présentation du Dataset

La base de données utilisée (*retail_customers_COMPLETE_CATEGORICAL.xlsx*) regroupe des informations riches et variées sur le parcours client, notamment :

- **Variables RFM** : Récence, Fréquence et Montant total des achats.
- **Données Démographiques** : Âge, pays de résidence, genre.
- **Comportement Digital** : Temps de connexion préféré, saison favorite, nombre de tickets de support ouverts.
- **Variable Cible** : La colonne **Churn** (0 pour fidèle, 1 pour parti).

Ce rapport détaille la démarche scientifique suivie, de la préparation rigoureuse des données jusqu'au déploiement d'une solution prédictive.

1 Prétraitement des Données (Preprocessing)

1.1 Introduction

La phase de prétraitement est une étape critique de notre pipeline. L'objectif est de transformer nos données brutes en une matrice mathématique compréhensible par nos algorithmes de Machine Learning, tout en évitant rigoureusement toute fuite de données (*Data Leakage*).

1.2 Nettoyage initial et Feature Engineering

Avant toute transformation mathématique, nous avons assaini la base de données :

- **Gestion des adresses IP (LastLoginIP)** : Les modèles ne pouvant traiter des chaînes de caractères brutes, nous avons appliqué du *Feature Engineering*. Nous avons créé une variable binaire `IP_IsPrivate` et extrait le premier octet de l'IP (`IP_FirstOctet`) pour capter un proxy géographique. La colonne d'origine a ensuite été supprimée.
- **Correction des anomalies temporelles** : La colonne `MonetaryTotal` contenait des dates parasites issues de saisies erronées. Celles-ci ont été identifiées et converties en valeurs manquantes (`NaN`) pour être traitées proprement.

1.3 Prévention de la Fuite de Données (Train/Test Split)

Justification de la Séparation

Une règle d'or de notre pipeline a été de diviser le jeu de données en un ensemble d'entraînement (Train - 80%) et un ensemble de test (Test - 20%) **avant** d'appliquer la moindre imputation ou normalisation. Si nous avions calculé des moyennes ou supprimé des colonnes sur l'ensemble global, des informations du jeu de test auraient influencé l'entraînement de notre modèle (*Data Leakage*).

1.4 Traitement des valeurs manquantes et aberrantes

Suite à la séparation, les opérations suivantes ont été apprises (*fit*) uniquement sur le Train, puis appliquées (*transform*) sur le Train et le Test :

- **Suppression des colonnes creuses** : Les variables présentant plus de 50% de valeurs manquantes ont été supprimées, car elles apportent plus de bruit statistique que de signal prédictif.
- **Clipping des valeurs aberrantes (*Outliers*)** : Les variables comme `SupportTicketsCount` ont été bornées pour éviter que des valeurs extrêmes ne faussent la variance de nos données.
- **Imputation par la médiane** : Pour les variables numériques contenant des `NaN`, nous avons choisi l'imputation par la médiane, celle-ci étant beaucoup moins sensible aux valeurs extrêmes que la moyenne.

1.5 Encodage des variables catégorielles

Les variables textuelles ont été numérisées selon leur nature :

- **Encodage Ordinal** : Utilisé pour les variables possédant une hiérarchie intrinsèque (ex : `LoyaltyLevel`). Nous avons assigné des valeurs croissantes (1, 2, 3...) pour respecter l'ordre naturel des catégories.
- **One-Hot Encoding** : Utilisé pour les variables nominales sans ordre logique (ex : `Gender`). Cela évite que l'algorithme ne déduise faussement qu'une catégorie est mathématiquement supérieure à une autre.
- **Target Encoding** : Pour la variable `Country`, le One-Hot Encoding aurait créé trop de colonnes. Nous avons donc remplacé chaque pays par son taux moyen de *Churn* observé dans le jeu d'entraînement.

1.6 Mise à l'échelle (Standardization)

Afin de mettre toutes les variables sur un pied d'égalité, nous avons appliqué un `StandardScaler`.

Justification Stratégique

Nous avons pris soin d'appliquer cette normalisation **uniquement sur les variables continues**. Les variables binaires (issues du One-Hot Encoding) n'ont pas été normalisées afin de préserver leur interprétabilité stricte (0 ou 1) et d'éviter de distordre leur poids statistique.

1.7 Conclusion

En conclusion, ce pipeline de prétraitement a permis de convertir une base de données brute et hétérogène en un jeu de données structuré et normalisé de (n) colonnes prédictives. En isolant strictement le jeu de test dès le départ et en adaptant chaque technique d'encodage à la nature spécifique des variables (Ordinale, One-Hot, Target), nous avons minimisé le risque de surapprentissage (*overfitting*). Maintenant, les données sont désormais prêtes pour la phase de **Modeling**.

2 Modélisation

2.1 Introduction

L'objectif de ce chapitre est de détailler la conception, l'entraînement et la validation du modèle prédictif de désabonnement (*Churn*). Après avoir préparé les données, nous avons dû faire face à des défis techniques majeurs, notamment le phénomène de *Data Leakage*, qui ont nécessité une révision profonde de notre approche pour garantir un modèle non seulement performant, mais surtout réaliste et exploitable en conditions réelles.

2.2 Choix de l'algorithme : Random Forest

Pour répondre à cette problématique de classification binaire, nous avons sélectionné l'algorithme **Random Forest** (Forêt Aléatoire). Ce choix repose sur trois piliers :

- **Robustesse** : En combinant les prédictions de multiples arbres de décision, il limite naturellement le risque de surapprentissage (*overfitting*).
- **Interprétabilité** : Il permet d'extraire l'importance des variables, offrant ainsi une vision claire des leviers business.
- **Non-linéarité** : Il capture efficacement les relations complexes entre les comportements d'achat et la fidélité.

2.3 Justification du paramétrage : Pourquoi 200 arbres ?

```
1  # Configuration du modèle
2  model = RandomForestClassifier(
3      n_estimators=200,          # 200 arbres de décision
4      max_depth=10,             # Profondeur max (evite le surapprentissage)
5      class_weight="balanced",  # Gere le desequilibre des classes
6      random_state=42,          # Reproductibilite
7      n_jobs=-1                 # Utilise toute la puissance CPU
8  )
9
10 print("Entrainement en cours...")
11 model.fit(X_train, y_train)
12 print("Entrainement termine !")
```

Listing 1 – Configuration et entraînement du modèle Random Forest

Nous avons configuré le modèle avec le paramètre `n_estimators=200`. Ce choix n'est pas arbitraire :

1. **Réduction de la variance** : 200 arbres permettent de stabiliser les votes de la forêt de manière bien plus efficace que la valeur par défaut (100), garantissant que le modèle ne soit pas sensible à de petites variations du jeu d'entraînement.
2. **Loi des rendements décroissants** : Au-delà de 200 arbres, l'amélioration de la performance devient négligeable, tandis que le coût computationnel (mémoire et temps de calcul) augmente de façon linéaire. C'est le **point d'équilibre optimal** pour notre dataset.

2.4 Le défi du Data Leakage (Fuite de données)

Une étape cruciale de ce projet a été l'identification d'une "triche" algorithmique. Lors des premiers tests, le modèle affichait un score parfait de 100%. Une telle performance trahissait la présence de *Data Leakage*.

Certaines variables comme `Recency`, `ChurnRiskCategory` ou `CustomerType` (contenant la catégorie "Perdu") donnaient indirectement la réponse au modèle. Pour assainir l'apprentissage, nous avons pris la décision radicale de supprimer l'intégralité de ces colonnes. Ce nettoyage a permis de passer d'un modèle "tricheur" à un modèle réellement prédictif basé sur le comportement client.

2.5 Évaluation et Résultats définitifs

Une fois le pipeline de données assaini, nous avons obtenu les résultats suivants, qui reflètent désormais la capacité réelle du modèle à généraliser sur de nouveaux clients.

```
=====
ENTRAÎNEMENT DU MODÈLE DE PRÉDICTION DU CHURN
=====
Dimensions - X_train: (3497, 69), X_test: (875, 69)

--- RÉSULTATS DE L'ÉVALUATION (SUR LE SET DE TEST) ---
precision    recall    f1-score    support

Fidèle       0.93      0.95      0.94      584
Churné       0.90      0.86      0.88      291

accuracy                    0.92      875
macro avg      0.91      0.90      0.91      875
weighted avg   0.92      0.92      0.92      875

Score ROC-AUC : 0.9689
Score F1 (Churn) : 0.8768
```

2.5.1 Validation Croisée et Stabilité

Pour garantir la fiabilité de ces chiffres, nous avons implémenté une validation croisée à 5 plis (*5-fold Cross-Validation*). Cette méthode entraîne et teste le modèle sur 5 découpes différentes du jeu de données pour vérifier sa stabilité.

Le code utilisé pour cette validation est le suivant :

```
1 cv_scores = cross_val_score(model, X_train, y_train,
2 cv=5, scoring="roc_auc", n_jobs=-1)
```

Listing 2 – Implémentation de la validation croisée

Les résultats obtenus (0.9653 ± 0.0058) confirment que le modèle est extrêmement stable et que ses performances ne sont pas dues au hasard du découpage des données.

2.6 Analyse de l'importance des variables

Le modèle identifie les facteurs clés suivants comme étant les plus prédictifs du départ d'un client :

- **FavoriteSeason_Automne (19.4%)** : La saisonnalité est le facteur prédominant.
- **PreferredMonth (14.1%)** : Confirme l'impact du calendrier sur l'attrition.
- **LoyaltyLevel (7.4%)** : L'ancienneté et le niveau d'engagement.
- **MonetaryTotal & TotalTransactions** : Le volume d'achat historique.

2.7 Conclusion du chapitre

En conclusion, ce chapitre a permis de transformer un modèle initialement biaisé par des fuites de données en un outil de prédiction robuste et fiable. Avec un rappel de 86% sur le Churn et une précision de 90%, l'entreprise dispose désormais d'un levier puissant pour identifier les clients à risque avant leur départ effectif. Le modèle, sauvegardé sous le nom `model.pkl`, est prêt à être intégré dans le processus décisionnel de l'équipe marketing.